

Ex81_spectrum_estimation

May 31, 2026

1 MSE StatDig : Chap 8 “Spectrum estimation”

1.1 Ex 8.1 Periodogram and modified periodogram

ver : DLY/31.05.2026

1.2 General

1.2.1 Description

The resolution of the periodogram, i.e. its ability to resolve closely spaced narrowband components is limited because of the Bartlett window.

Since the width of the main lobe increases as N decreases, for a given N there is a limit on how closely two sinusoids may be located before they can no longer be resolved.

This is usually defined as the bandwidth of the window at its half power points (-6dB), which is for the Bartlett window at:

$$Res\{\hat{P}_{per}(e^{j\omega})\} = 0.89 \frac{2\pi}{N}$$

This relation is valid for $\omega \in [-\pi \dots \pi]$.

In the frequency domain, $f \in [-Fe/2 \dots Fe/2]$, the bandwidth of the window at its half power points (-6dB) is:

$$Res\{\hat{P}_{per}(e^{j\omega})\} = 0.89 \frac{Fe}{N}$$

1.2.2 Example:

Let's take an example where N is too low to separate 2 close sinusoids.

In this example, the signal is a mixture of 2 sinusoids with $\omega = [0.4\pi, 0.45\pi]$, with random phase and added Gaussian noise $v[n]$ (with $\mu = 0$, $\sigma = 5$):

$$x[n] = A_1 \sin(n\omega_1 + \phi_1) + A_2 \sin(n\omega_2 + \phi_2) + v[n]$$

To evaluate the effect on the resolution, the number of points N will be changed: $N = [32; 128]$

The process must be realized $Nr = 50$ times.

1.2.3 Work

Ex1 : Periodogram

Ex1.1 : Signal generation

- Generate the random process signals with $N = 32$ and $N = 128$ for $N_r = 50$
- Plot the signals in the time domain

For the generation of the signal, you can use the following code:

```
import numpy as np
import matplotlib.pyplot as plt

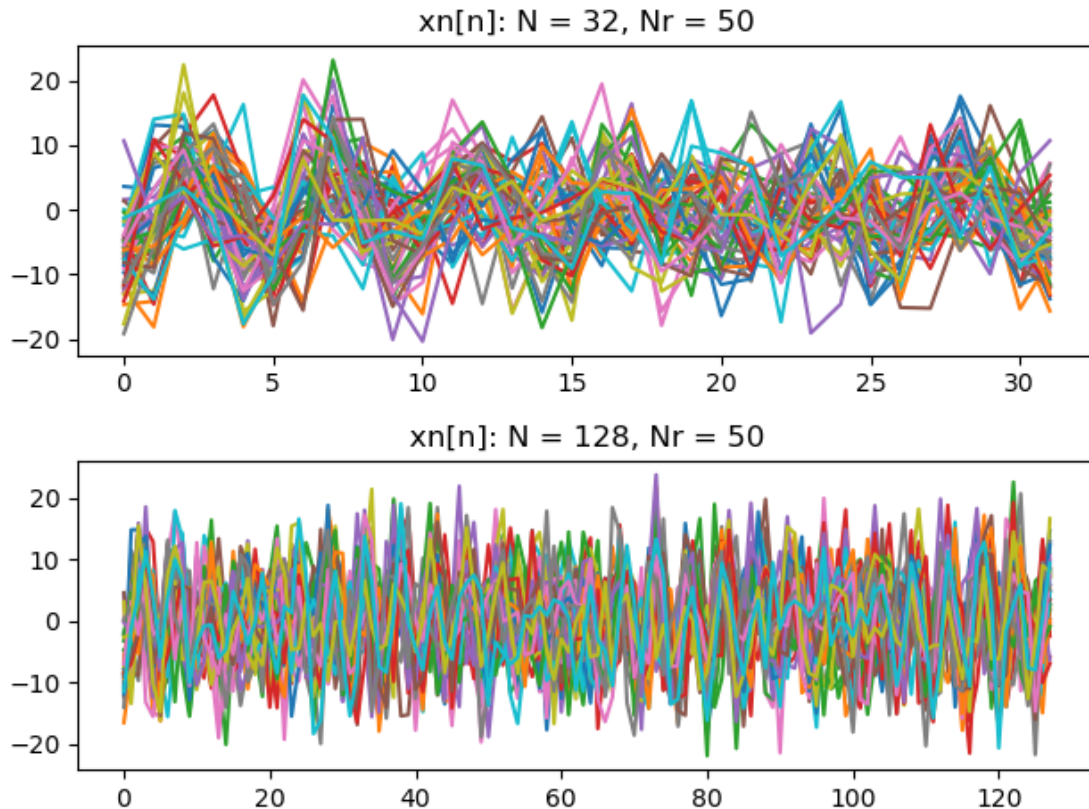
Npoints = np.array([32,128])
omegas = np.array([0.4*np.pi,0.45*np.pi])
sigmaV = 5
As = [5, 5]
Nr = 50
NFFT = 512
NFFT2 = int(NFFT/2)

np.random.seed(0)

xn_all = []
for i in range(len(Npoints)):
    N = Npoints[i]
    n = np.arange(N)[: ,None]
    nn = np.repeat(n,Nr,1).T

    v = sigmaV*np.random.randn(Nr,N)
    Phi = np.random.rand(Nr,len(omegas)) * np.pi - np.pi
    x = v.copy()
    for k in range(len(omegas)):
        x += As[k]*np.sin(nn*omegas[k]+Phi[:,k] [: ,None])
    xn_all.append(x)

(50, 32)
(50, 128)
```



Ex1.2 : Periodogram computation The goal is now to compute the periodogram for each realization of each random process and to display it.

Your work :

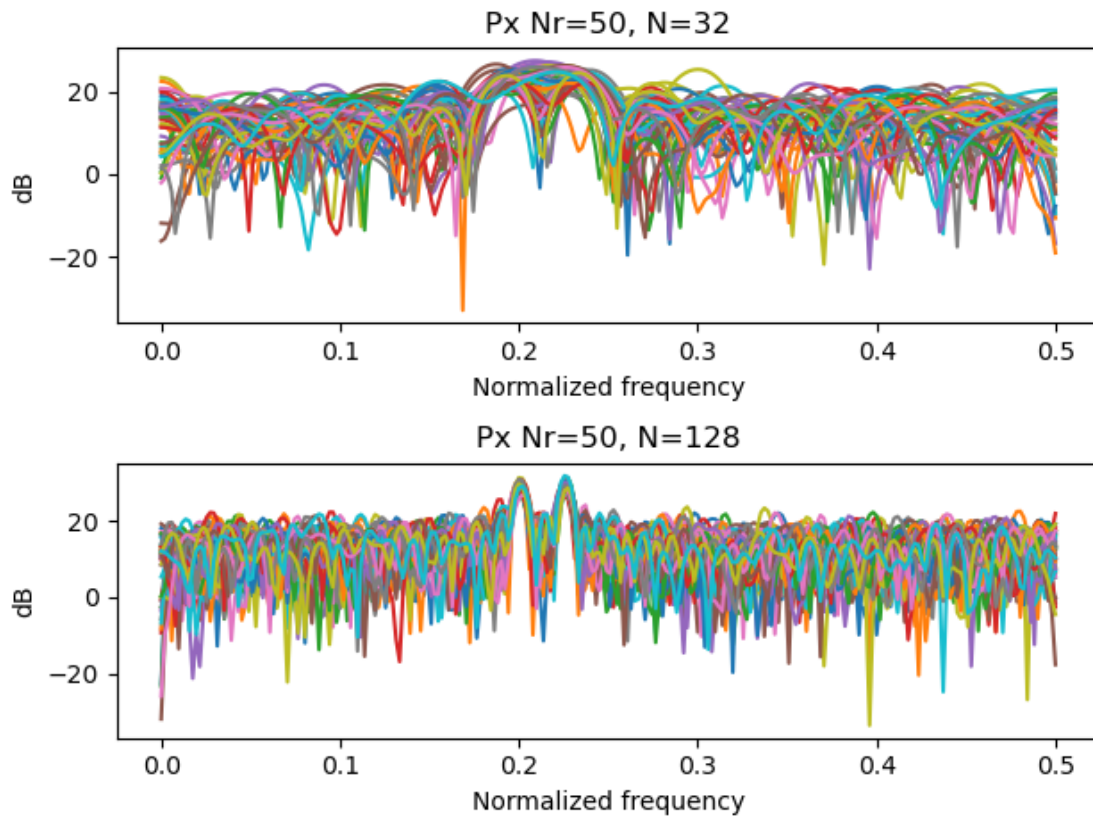
- Implement the function `myPer` which computes the periodogram of a signal.

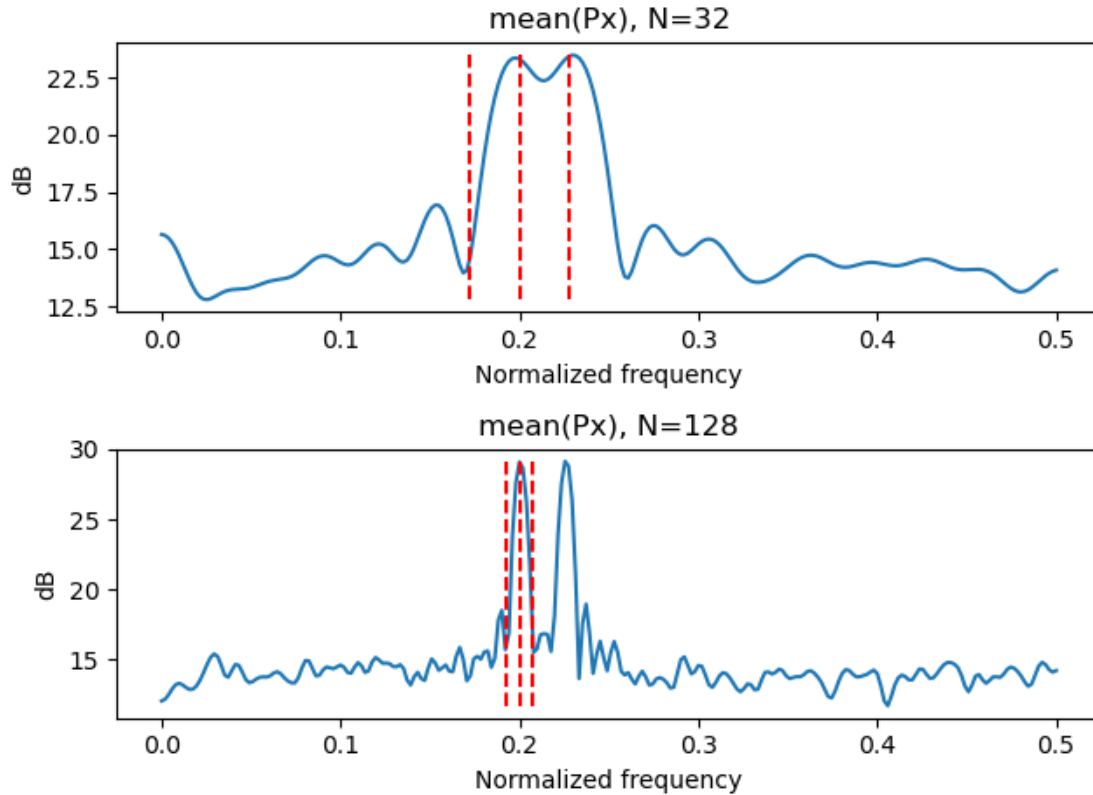
```
def myPer(x, NFFT):
    '''
    MYPER    Estimate the spectrum of a process using the periodogram
    USAGE    Px = myPer(x,NFFT)
             The spectrum of a process x is estimated using the periodogram.
    INPUT:
        x      : (N,) input sequence
        NFFT   : scalar number of points used for the DTFT
    OUTPUT:
        Px     : (Nfft,) The periodogram is returned in Px using a linear scale.
    '''

    # YOUR CODE HERE
```

```
return Px
```

- Display each periodogram on two separates figures (one for each N).
- Display the average periodogram of each realization on two separates figure (one for each N).





Ex2 : Modified Periodogram Instead of applying a rectangular window to $x[n]$ to make it finite length, other kind of windows could be used. This degree of freedom can be used to modulate the amount of smoothing to be applied to the periodogram. Although a rectangular window has a narrow main lobe compared to other windows and therefore, produces the least amount of spectral smoothing, it has relatively large sidelobes that may lead to the masking of weak narrowband components.

Ex2.1 : Signal generation Lets start to generate the same process than before but with different values: $\omega = [0.25\pi, 0.3\pi]$ with random phase and with added Gaussian noise $v[n]$ ($\mu = 0, \sigma = 0.3$)

$$x[n] = A_1 \sin(n\omega_1 + \phi_1) + A_2 \sin(n\omega_2 + \phi_2) + v[n]$$

Where the amplitude is $A_1 = 0.1$ and $A_2 = 1$. Only $N=256$ samples are generated for each of the 20 realizations.

- Display each realization of the random process in the time domain on a single figure.

```
A = [0.1, 1]
N3 = 256
Nr = 20
n3 = np.arange(N3)
NFFT = 2048
```

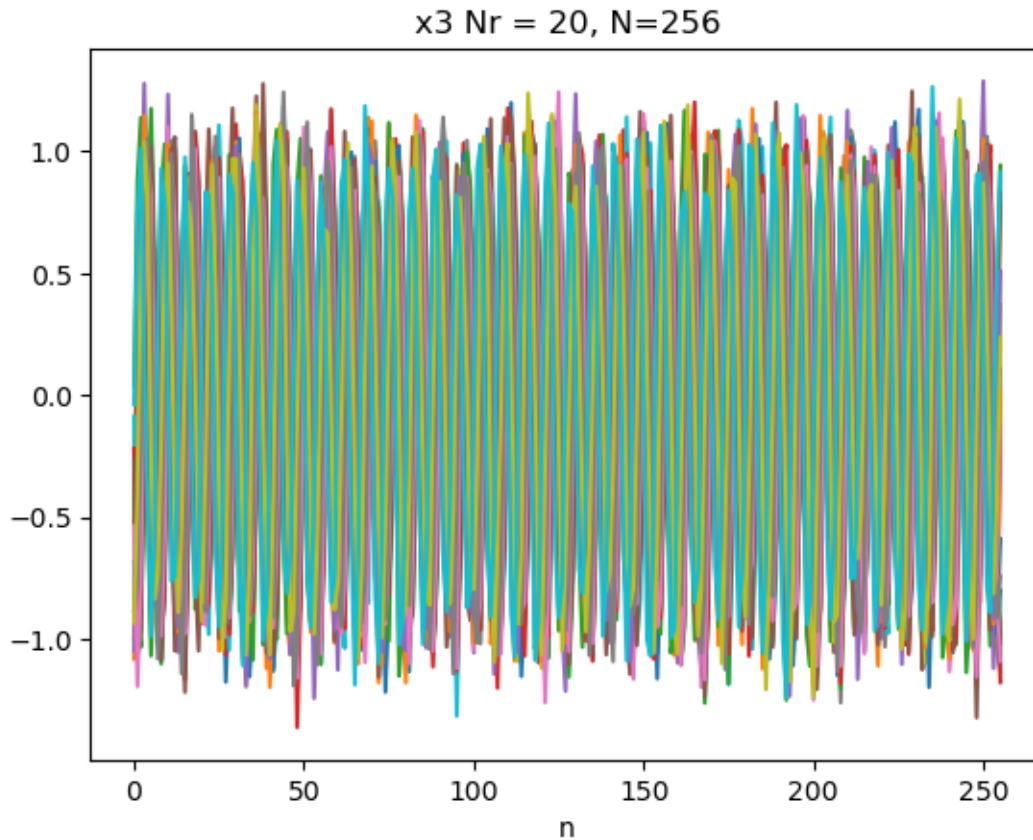
```

NFFT2 = int(NFFT//2)
sigmaV = .3
omega = np.array([.25, .3]) * np.pi

n = np.arange(N3)[: ,None]
nn = np.repeat(n,Nr,1).T

V3 = sigmaV * sigmaV*np.random.randn(Nr,N3)
Phi3 = np.random.rand(Nr,2) * np.pi - np.pi
x3 = A[0] * np.sin(nn*omega[0]+Phi3[:,0][: ,None])+A[1]*np.sin(nn*omega[1]+Phi3[:,1][: ,None]) +

```



Ex2.2 : Windows analysis The modified periodogram can use different windows. For this example, we will use only hamming window instead of the rectangular window. As the spectrum will be a convolution with the square fourier transform of the window, it is interesting to look the window spectrum. To achieve that:

- Create the rectangular and hamming windows using N points
- Normalize each window by their respective rms value
- Compute the DTFT using NFFT points
- Display each window in the time domain and the DTFT of each window in the frequency domain

The rectangular window is easily computed using `np.ones((N))`. The hamming window is easily generated with `wh=hamming(N)`. Below, the two windows are compared:

```
C:\Users\david.lavanchy\AppData\Local\Temp\ipykernel_34440\2626534360.py:24:
```

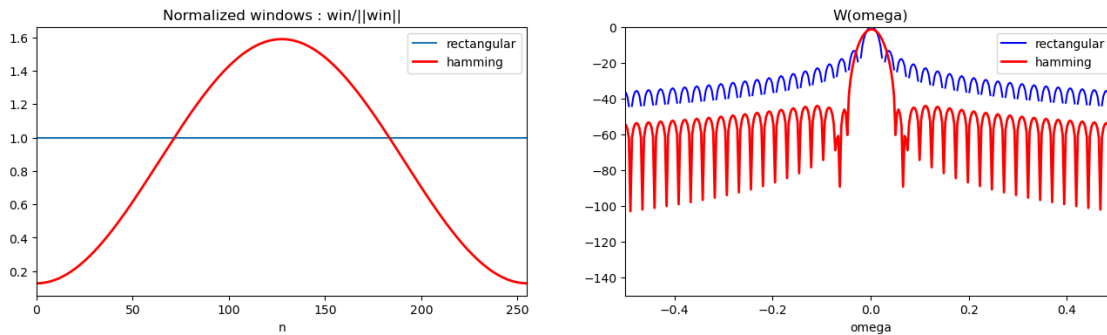
```
RuntimeWarning: divide by zero encountered in log10
```

```
plt.plot(wf,20*np.log10(abs(WR)),'b',label='rectangular')
```

```
C:\Users\david.lavanchy\AppData\Local\Temp\ipykernel_34440\2626534360.py:25:
```

```
RuntimeWarning: divide by zero encountered in log10
```

```
plt.plot(wf,20*np.log10(abs(WH)),'r',linewidth=2,label='hamming')
```



2.3 : Estimated power spectrum density The periodogram and modified periodogram (hamming) are used to estimate the power spectrum density of the realizations. To build the figure, you must start to design the function `myMper()`

- Implement the function `myMper()` which computes the modified periodogram of a signal with a hamming window

```
def myMper(x, NFFT):
```

```
'''
```

```
MYPER    Estimate the spectrum of a process using the modified periodogram
```

```
USAGE    Px = myMper(x,NFFT)
```

```
    The spectrum of a process x is estimated using the modified periodogram.
```

```
INPUT:
```

```
    x      : (N,) input sequence
```

```
    NFFT   : scalar number of points used for the DTFT
```

```
OUTPUT:
```

```
    Px     : (Nfft,) The modified periodogram is returned in Px using a linear scale.
```

```
'''
```

```
# YOUR CODE HERE
```

```
return Px
```

- Compute the periodogram and modified periodogram of each realization
- Compute the average periodogram and modified periodogram of each realization
- Display the periodogram and modified periodogram of each realization and the average periodogram

